

Kern

Observable Event Database - Technical Specification

Version 0.2

Java 25+ | RocksDB | Spring Boot | Armeria | gRPC

Group ID: it.riccisi.kern

Document control

Field	Value
Status	Implementation baseline
Supersedes	Weave Event Store Specification v0.1
Primary change	Query-based DCB consistency replaces key-revision DCB
Product name	Kern
Maven groupId	it.riccisi.kern
Audience	Architects, implementers, reviewers and AI coding agents
Normative statement This document is the authoritative specification for Kern v0.2. The migration document describes how to move existing v0.1 code to this model.	

1. Purpose, principles and scope

1.1 Purpose

Kern is a standalone event database. It provides immutable event persistence, globally ordered positions, query-based conditional append, subscriptions and diagnostics. It is independent of any decision framework and does not know commands, decisions, invariants, aggregates or domain objects.

1.2 Architectural principles

- One authoritative append-only event log.
- An event may be associated with multiple tags and therefore participate in multiple histories without duplication.
- Consistency is expressed as EventQuery + observed global SequencePosition.
- Traditional aggregate-centric Event Sourcing is a constrained form of the same query model.
- RocksDB provides physical storage; Kern provides event semantics, protocols, consistency and operations.
- Observability and explainability are product capabilities, not optional logging.
- The v1 production target is a durable single-node server with one logical append writer.

1.3 Non-goals

- No decision runtime or business-rule execution.
- No arbitrary SQL-like query language.
- No multi-node consensus or automatic failover in v1.
- No server-side domain projections in v1.
- No interpretation of event payloads.

2. Unified event and consistency model

2.1 Event

```
public record EventData(  
    EventId id,  
    EventType type,  
    Set<EventTag> tags,  
    ContentType contentType,  
    byte[] payload,  
    byte[] metadata  
) {}
```

Events are immutable. Tags are unordered logical associations such as student:S1, course:C1 or order:O123. Kern does not impose aggregate ownership and does not duplicate an event when it has multiple tags.

2.2 Stored event

```
public record SequencedEvent(  
    SequencePosition position,  
    Instant recordedAt,  
    EventData event  
) {}
```

2.3 Query semantics

```
public record EventQuery(List<QueryItem> items) {}  
  
public record QueryItem(  
    Set<EventType> types,  
    Set<EventTag> tags  
) {}
```

A `QueryItem` matches an event when the type constraint matches and the event contains every required tag. An empty type set means all event types. Query items are OR-combined; constraints inside an item are AND-combined.

```
boolean matches(QueryItem item, EventData event) {  
    boolean typeMatches = item.types().isEmpty()  
        || item.types().contains(event.type());  
    boolean tagsMatch = event.tags().containsAll(item.tags());  
    return typeMatches && tagsMatch;  
}  
  
boolean matches(EventQuery query, EventData event) {  
    return query.items().isEmpty()  
        || query.items().stream().anyMatch(item -> matches(item, event));  
}
```

2.4 Observation

```
public record QueryResult(  
    List<SequencedEvent> events,  
    SequencePosition observedAt,  
    ContinuationToken continuation  
) {}
```

`observedAt` is the global log head read from the same RocksDB snapshot used for the query. It may be greater than the position of the last matching event. It means that all matching events up to that global point were observed.

2.5 Conditional append

```
public record AppendCondition(  
    EventQuery failIfEventsMatch,  
    SequencePosition after  
) {}
```

The append succeeds only if no event with position greater than after matches the complete query. Both types and tags participate in conflict detection.

2.6 Traditional Event Sourcing as a special case

A classic aggregate stream is represented by one stable tag and no type filter. Any later event with the same tag invalidates the observation.

```
EventTag aggregate = new EventTag("course", "C1");  
EventQuery streamQuery = new EventQuery(List.of(  
    new QueryItem(Set.of(), Set.of(aggregate))  
));
```

Important

Stream revision may be exposed as a convenience compatibility feature, but it is not the primary consistency model. The authoritative primitive is query + global observed position.

3. DCB example: student enrolment

The enrolment decision needs course capacity, student existence and enrolment facts, but not address or description changes.

```
EventQuery query = new EventQuery(List.of(  
    new QueryItem(  
        Set.of(COURSE_DEFINED, COURSE_CAPACITY_CHANGED),  
        Set.of(tag("course", "C1"))  
    ),  
    new QueryItem(  
        Set.of(STUDENT_REGISTERED),  
        Set.of(tag("student", "S1"))  
    ),  
    new QueryItem(  
        Set.of(STUDENT_ENROLLED, STUDENT_WITHDREW),  
        Set.of(tag("course", "C1"))  
    ),  
    new QueryItem(  
        Set.of(STUDENT_ENROLLED, STUDENT_WITHDREW),  
        Set.of(tag("student", "S1"))  
    )  
));
```

The produced event is stored once and tagged for both histories:

```
new EventData(  
    EventId.random(),  
    STUDENT_ENROLLED,  
    Set.of(tag("student", "S1"), tag("course", "C1")),  
    ContentType.JSON,  
    payload,  
    metadata  
);
```

Concurrent event after observedAt	Matches condition?	Result
StudentAddressChanged [student:S1]	No - excluded type	Append remains valid
CourseDescriptionChanged [course:C1]	No - excluded type	Append remains valid
StudentEnrolled [student:S2, course:C1]	Yes - enrolment + course:C1	Conflict
StudentEnrolled [student:S1, course:C9]	Yes - enrolment + student:S1	Conflict

4. System architecture

Module	Responsibility
kern-api	Public immutable Java model and errors
kern-core	Query engine contracts, append coordination, subscriptions and diagnostics
kern-rocksdb	RocksDB lifecycle, codecs, indices, backup and recovery
kern-protocol	Protobuf contracts and protocol mappings
kern-armeria	gRPC/HTTP data plane
kern-spring-boot	Bootstrap, configuration, Actuator and control plane
kern-client	Java client, retry and streaming
kern-testkit	Fixtures, conformance tests and fault injection helpers

4.1 Runtime topology

Armeria event loops

- > bounded append queue
- > single logical AppendCoordinator
- > conflict detection
- > RocksDB WriteBatch + WAL sync
- > completion of client futures

Read and subscription workloads

- > dedicated read executors
- > RocksDB snapshots and iterators

4.2 Framework boundaries

- Spring Boot is used for lifecycle, configuration, health, security and administration.
- Armeria provides the low-latency gRPC/HTTP data plane.
- No Spring, Reactor or transport types are allowed in kern-api, kern-core or storage contracts.
- RocksDB calls never block Netty event-loop threads.

5. Public API

5.1 Core Java API

```
public interface EventStore {
    QueryResult read(ReadRequest request);

    CompletionStage<AppendResult> append(AppendRequest request);

    EventCursor openCursor(CursorRequest request);
}

public record AppendRequest(
    Namespace namespace,
    List<EventData> events,
    AppendCondition condition,
    IdempotencyKey idempotencyKey
) {}
```

5.2 Read request

```
public record ReadRequest(  
    Namespace namespace,  
    EventQuery query,  
    SequencePosition fromExclusive,  
    Integer limit,  
    ContinuationToken continuation  
) {}
```

5.3 gRPC service

```
service KernEventStore {  
    rpc Read(ReadRequest) returns (ReadResponse);  
    rpc Append(AppendRequest) returns (AppendResponse);  
    rpc Observe(ObserveRequest) returns (stream SequencedEvent);  
    rpc Head(HeadRequest) returns (HeadResponse);  
}
```

5.4 Error model

Code	Meaning	Retry
QUERY_CONFLICT	A later matching event invalidated the condition	Yes, after reread
IDEMPOTENCY_MISMATCH	Same key used with a different request fingerprint	No
EVENT_ALREADY_EXISTS	Event ID already stored with different content	No
OVERLOADED	Bounded append queue full	Yes, with backoff
STORAGE_UNAVAILABLE	RocksDB or disk unavailable	Yes depending on health
INVALID_QUERY	Unsupported or malformed query	No

6. RocksDB physical model

6.1 Column families

Column family	Authoritative?	Key -> value
events	Yes	namespace + global position -> encoded event
event_ids	Derived uniqueness index	namespace + event UUID -> position + fingerprint
tag_type_index	Derived	namespace + tag + type + position -> empty/reference
type_index	Derived	namespace + type + position -> empty/reference
tag_index	Derived	namespace + tag + position -> empty/reference
idempotency	Yes for retry semantics	namespace + key -> request fingerprint + append result
metadata	Yes	namespace + head/database format/shutdown marker -> value
diagnostics	Optional durable diagnostics	append ID -> diagnostic record

The v0.1 consistency and tag-watermark column families are removed from the normative design. A watermark can exist only as a non-authoritative optimization and must never replace query evaluation.

6.2 Key encoding

- All numeric components use fixed-width big-endian encoding.
- Lengths or zero-safe escaping delimit variable binary fields.
- Every key begins with a namespace prefix.
- Key codec version is persisted in metadata and covered by compatibility tests.

```
// Conceptual order, not textual production encoding
[tag namespace][tag name][tag value][event type][position:8]
[position:8] -> StoredEvent
```

6.3 Event record format

- Format version
- Event ID
- Global position
- Recorded timestamp
- Type
- Tag count and tags
- Content type
- Payload length and bytes
- Metadata length and bytes
- Checksum

6.4 Atomic batch

A successful append writes every event, all indices, event-ID uniqueness records, idempotency outcome and the new head in one RocksDB WriteBatch with WAL sync enabled by default.

7. Query execution

7.1 Snapshot invariant

```
public QueryResult read(ReadRequest request) {
    Snapshot snapshot = db.getSnapshot();
    try (ReadOptions options = new ReadOptions().setSnapshot(snapshot)) {
        SequencePosition observedAt = metadata.readHead(options);
        List<SequencedEvent> events = queryEngine.execute(
            request.query(), request.fromExclusive(), observedAt,
            request.limit(), options);
        return new QueryResult(events, observedAt, continuation(...));
    } finally {
        db.releaseSnapshot(snapshot);
    }
}
```

7.2 Query planning

For each QueryItem, the engine chooses the most selective available posting list, scans positions in order and verifies remaining constraints against the event record or auxiliary indices. Results from OR items are merged and de-duplicated by global position.

```
public interface QueryPlanner {
    QueryPlan plan(EventQuery query, IndexStatistics statistics);
}

sealed interface QueryPlan permits TagTypeScan, TagScan,
    TypeScan, FullLogScan, UnionPlan {}
```

7.3 Multi-tag AND

For an item containing several tags, Kern scans one candidate index and verifies that each candidate event contains all required tags. Future versions may use posting-list intersection. Correctness must not depend on selectivity statistics.

7.4 Empty query

An empty query means all events. It is valid for global subscriptions and administrative scans, but may be forbidden for conditional append unless explicitly enabled because it serializes all writers against the global log head.

8. Append protocol

1. Validate request limits, tag syntax and query complexity.
2. Calculate deterministic request fingerprint.
3. Check idempotency record and event-ID uniqueness.
4. On the single writer, evaluate the complete condition after the supplied position.
5. If a matching event exists, produce an explainable QueryConflict and write nothing.
6. Assign contiguous global positions to the new events.
7. Build one WriteBatch containing events, indices, IDs, idempotency record, diagnostics and head.
8. Write with WAL sync according to durability policy.
9. Publish the new high-water mark to waiting subscriptions.
10. Complete client futures and tracing spans.

8.1 Conflict detection

```
public Optional<QueryConflict> detect(AppendCondition condition,
                                     ReadOptions readOptions) {
    return queryEngine.findFirst(
        condition.failIfEventsMatch(),
        condition.after().next(),
        metadata.readHead(readOptions),
        readOptions
    ).map(event -> QueryConflict.from(condition, event));
}
```

8.2 Explainable conflict

```
public record QueryConflict(
    SequencePosition observedAt,
    SequencePosition conflictingPosition,
    EventId conflictingEventId,
    EventType conflictingType,
    Set<EventTag> conflictingTags,
    int matchedQueryItem
) {}
```

8.3 Group commit

Requests may share one physical WriteBatch only when they are validated sequentially against the evolving in-memory batch state. The coordinator must make earlier events in the same group visible to conflict checks for later requests before the batch is committed.


```

for (PendingAppend pending : group) {
    Conflict conflict = overlay.detect(pending.condition());
    if (conflict != null) reject(pending, conflict);
    else overlay.apply(prepare(pending));
}
storage.write(overlay.toWriteBatch(), SYNC_WAL);

```

9. Subscription and cursors

- Observe streams events after a global position.
- Apply the same EventQuery semantics used for reads and consistency.
- Use bounded per-subscriber buffers and gRPC flow control.
- Wake-ups are hints; recovery always resumes by position from RocksDB.
- The first release keeps checkpoints in consumers; durable server-side subscriptions are roadmap work.

```

public record CursorRequest(
    Namespace namespace,
    EventQuery query,
    SequencePosition after,
    int batchSize
) {}

```

10. Idempotency and duplicate protection

An idempotency key maps to a fingerprint of the complete logical request, including events and append condition. Reuse with the same fingerprint returns the original result; reuse with a different fingerprint is rejected.

```

public record IdempotencyRecord(
    RequestFingerprint fingerprint,
    AppendResult result,
    Instant committedAt
) {}

```

11. Durability, recovery and maintenance

11.1 Durability

- WAL enabled and sync=true by default.
- No acknowledgement before successful durable write.
- Configurable relaxed mode only for tests or explicitly accepted workloads.
- Disk-full and background-error states immediately fail readiness and reject writes.

11.2 Startup

11. Open RocksDB and required column families.
12. Allow WAL recovery.
13. Validate format and codec versions.
14. Verify head against the last event key.
15. Validate or mark derived indices for repair.
16. Start coordinator and readers.
17. Expose readiness only after all checks pass.

11.3 Verify and rebuild

```
public interface IntegrityService {  
    VerificationReport verify(VerificationScope scope);  
    RebuildResult rebuild(IndexName index, SequencePosition from);  
}
```

events is the authoritative event history. tag, type and combined indices must be rebuildable from it. Idempotency records are operationally authoritative and require backup rather than blind reconstruction.

12. Observability and debugging

12.1 Product objective

Kern must make Event Sourcing and DCB failures understandable. Every conflict should answer which query item matched, which event invalidated the observation and how long the stale observation existed.

12.2 Metrics

Metric group	Examples
Append	duration, queue wait, events/request, batch size, WAL sync, rejected total
DCB	query conflicts, conflict age, items evaluated, candidate scans, false-candidate checks
Read	query latency, scanned index entries, returned events, snapshot lifetime
Subscription	connections, lag, buffered events, slow-consumer disconnects
RocksDB	memtable bytes, block cache, L0 files, pending compaction bytes, stalls, live data size
Operations	backup age, last verified position, disk free, startup recovery time

12.3 Tracing

```
kern.append  
kern.idempotency.lookup  
kern.condition.plan  
kern.condition.scan  
kern.batch.prepare  
kern.rocksdb.write  
kern.wal.sync
```

```
Attributes:  
kern.namespace  
kern.observed_at  
kern.head_before  
kern.events_count  
kern.query_items  
kern.conflicting_position (on rejection)
```

12.4 Diagnostic API

- Explain an append conflict using a diagnostic ID.
- Inspect an event by ID or position.
- Execute a query in dry-run mode and show the selected plan.
- Show current head and namespace statistics.
- Show consumer lag and RocksDB health.
- Export a bounded diagnostics snapshot for incident analysis.

```

public record ConflictExplanation(
    AppendId appendId,
    EventQuery condition,
    SequencePosition observedAt,
    SequencedEvent conflictingEvent,
    int matchedItem,
    Duration observationAge,
    QueryPlanSummary plan
) {}

```

13. Configuration and security

```

kern:
  data-directory: /var/lib/kern
  append:
    queue-capacity: 65536
    max-group-requests: 256
    max-group-bytes: 4194304
    sync-wal: true
  query:
    max-items: 32
    max-tags-per-item: 8
    max-types-per-item: 64
    default-page-size: 500
  subscriptions:
    max-buffered-events: 1024
  diagnostics:
    persist-append-records: true

```

- TLS for network endpoints.
- Authentication on data and control planes.
- Namespace-scoped authorization.
- Administrative APIs separated by port and role.
- Payloads treated as opaque sensitive data and excluded from logs by default.

14. Testing strategy

14.1 Conformance properties

- No successful append is partially visible.
- Global positions are unique, contiguous for one committed request and monotonically increasing.
- A conditional append is rejected iff a later event matches the complete query.
- Irrelevant event types sharing a tag do not cause conflicts.
- A multi-tag event is returned by every applicable tag query but stored once.
- Classic single-tag/no-type queries behave like aggregate optimistic concurrency.
- Read results contain every matching event at or below observedAt.
- Idempotent retry never writes another event.

14.2 Mandatory race tests

Test	Expected result
Address change races with enrolment	No conflict
Course description change races with enrolment	No conflict
Another student enrolls in same course	Conflict
Same student enrolls in another course	Conflict
Two appends in same group affect same query	Second sees first through overlay and conflicts

Test	Expected result
Crash after RocksDB write before response	Retry returns same result

14.3 Fault injection

- Kill before and after WAL sync.
- Disk-full simulation.
- Corrupt derived index and rebuild.
- Slow fsync and queue saturation.
- Subscriber disconnect and resume.
- Codec-version incompatibility.

15. Packaging and deployment

- Executable Spring Boot application kern-server.
- RocksDB native binaries packaged through rocksdbjni.
- Docker image with non-root user and persistent volume.
- Separate data-plane and management ports.
- Kubernetes StatefulSet supported, but one active writer per database directory.
- Backups copied to external durable storage.

16. Roadmap

Phase	Capabilities
0.2 implementation	Unified query model, DCB conditional append, classic adapter, diagnostics
0.3	Persistent subscriptions, dead-letter handling, richer query planner statistics
0.4	Stream snapshots as opaque records, online index rebuild, schema registry hooks
0.5	Follower replication and read replicas, replication observability
1.x	Leader election, automatic failover, partitioned log with explicit consistency constraints
Later	Storage tiering, object-store archive, optional posting-list intersection and adaptive planning

17. Acceptance criteria for v0.2

- The old key-revision DCB path is absent from public APIs.
- The student/course examples pass all race scenarios.
- Traditional stream API is implemented as an adapter over the query model.
- All append mutations are in one synced WriteBatch.
- Conflict responses identify the actual invalidating event and matched query item.
- Metrics and traces expose query planning, scan cost and WAL latency.
- Recovery, backup and index rebuild are exercised in automated tests.

- Core modules compile without Spring or Armeria dependencies.

Appendix A - Suggested package structure

```

it.riccisi.kern.api
  event
  query
  append
  error

it.riccisi.kern.core
  append
  query
  subscription
  diagnostics

it.riccisi.kern.rocksdb
  codec
  key
  index
  lifecycle
  backup
  recovery

it.riccisi.kern.server
  grpc
  http
  configuration
  security
  actuator

```

Appendix B - Naming rules

Concept	Required term	Avoid
Global order	SequencePosition	event revision for global position
Read marker	observedAt	aggregate version
DCB protection	AppendCondition(EventQuery, after)	consistency key revisions
Association	EventTag	aggregate ownership
Conflict	QueryConflict	optimistic lock exception without details